

# Lecture 4

## Greedy Algorithms



# Announcements

- ❖ Lab 1 on next Monday
  - Empirically benchmarking algorithms
- ❖ Office hours are up on course website
  - T: 12pm-4pm; F: 11am-1pm
  - Schedule an appointment individually or as a group

# Big Picture

- ❖ **This week:** start thinking algorithmically
  - Greedy algorithms & empirical benchmarking
  - F: Project 1 topics released
- ❖ **Next week:** running time analysis
  - Growth rates & Big-O
  - F: Project 1 Proposal Due

# Today

- ❖ Largest Pair Problem
  - Problem statement
  - Baseline algorithm
  - Greedy algorithm
  - Which is better?
- ❖ Revisit interval scheduling problem

# Largest Pair Problem

- ❖ Given a list of  $n$  numbers, find a pair that has the greatest sum

# Largest Pair Problem

- ❖ Given a list of  $n$  numbers, find a pair that has the greatest sum

12, -4, 3, 4, 0, 4

# Largest Pair Problem

- ❖ Given a list of  $n$  numbers, find a pair that has the greatest sum

12, -4, 3, 4, 0, 4

- ❖ Returns pair 12, 4 with sum 16

# Baseline Algorithm

- ❖ What is a simple algorithm that always correctly solves this problem?

# Baseline Algorithm

- ❖ What is a simple algorithm that always correctly solves this problem?
  - Try every possible pair and see which one has the largest sum

# Baseline Algorithm

- ❖ What is a simple algorithm that always correctly solves this problem?
  - Try every possible pair and see which one has the largest sum
- ❖ Why is it guaranteed to work?

# Baseline Algorithm

- ❖ What is a simple algorithm that always correctly solves this problem?
  - Try every possible pair and see which one has the largest sum
- ❖ Why is it guaranteed to work?
  - Suppose it does not find the pair with the largest sum. Then there is some pair with a larger sum. However, this pair must have not been checked. This is a contradiction, because the baseline method checks every pair.

# Baseline Algorithm

- ❖ What is a simple algorithm that always correctly solves this problem?
  - Try every possible pair and see which one has the largest sum
- ❖ Why is it guaranteed to work?
  - Suppose it does not find the pair with the largest sum. Then there is some pair with a larger sum. However, this pair must have not been checked. This is a contradiction, because the baseline method checks every pair.
- ❖ How many pairs does the baseline method need to try?

# Baseline Algorithm

- ❖ What is a simple algorithm that always correctly solves this problem?
  - Try every possible pair and see which one has the largest sum
- ❖ Why is it guaranteed to work?
  - Suppose it does not find the pair with the largest sum. Then there is some pair with a larger sum. However, this pair must have not been checked. This is a contradiction, because the baseline method checks every pair.
- ❖ How many pairs does the baseline method need to try?

$$\binom{n}{2} = \frac{n!}{2!(n-2)!} = \frac{n(n-1)}{2}$$

# Greedy Algorithm Approach

- ❖ Idea: iteratively choose the thing that looks the best
- ❖ Recipe: sort items into some "natural order" and choose them one-by-one as long as they are compatible with the items already chosen

# Greedy Algorithm

- ❖ Sort the list of numbers in descending order and select the first two numbers

# Greedy Algorithm

- ❖ Sort the list of numbers in descending order and select the first two numbers
- ❖ Why is this guaranteed to work?

# Greedy Algorithm

- ❖ Sort the list of numbers in descending order and select the first two numbers
- ❖ Why is this guaranteed to work?
  - Suppose it fails. Let  $x$  and  $y$  be the first two numbers in the list. Then, there must be another pair of numbers  $w$  and  $z$  such that  $x + y < w + z$ . Either  $x < w$  or  $y < z$ . This is a contradiction, since this method sorts the list in descending order.

# Greedy Algorithm

- ❖ Sort the list of numbers in descending order and select the first two numbers
- ❖ Why is this guaranteed to work?
  - Suppose it fails. Let  $x$  and  $y$  be the first two numbers in the list. Then, there must be another pair of numbers  $w$  and  $z$  such that  $x + y < w + z$ . Either  $x < w$  or  $y < z$ . This is a contradiction, since this method sorts the list in descending order.
- ❖ How many steps does this algorithm require?

# Greedy Algorithm

- ❖ Sort the list of numbers in descending order and select the first two numbers
- ❖ Why is this guaranteed to work?
  - Suppose it fails. Let  $x$  and  $y$  be the first two numbers in the list. Then, there must be another pair of numbers  $w$  and  $z$  such that  $x + y < w + z$ . Either  $x < w$  or  $y < z$ . This is a contradiction, since this method sorts the list in descending order.
- ❖ How many steps does this algorithm require?
  - Cost of sorting plus 2
  - Efficient sorting takes on the order of  $n \cdot \log n$  steps (Mergesort)

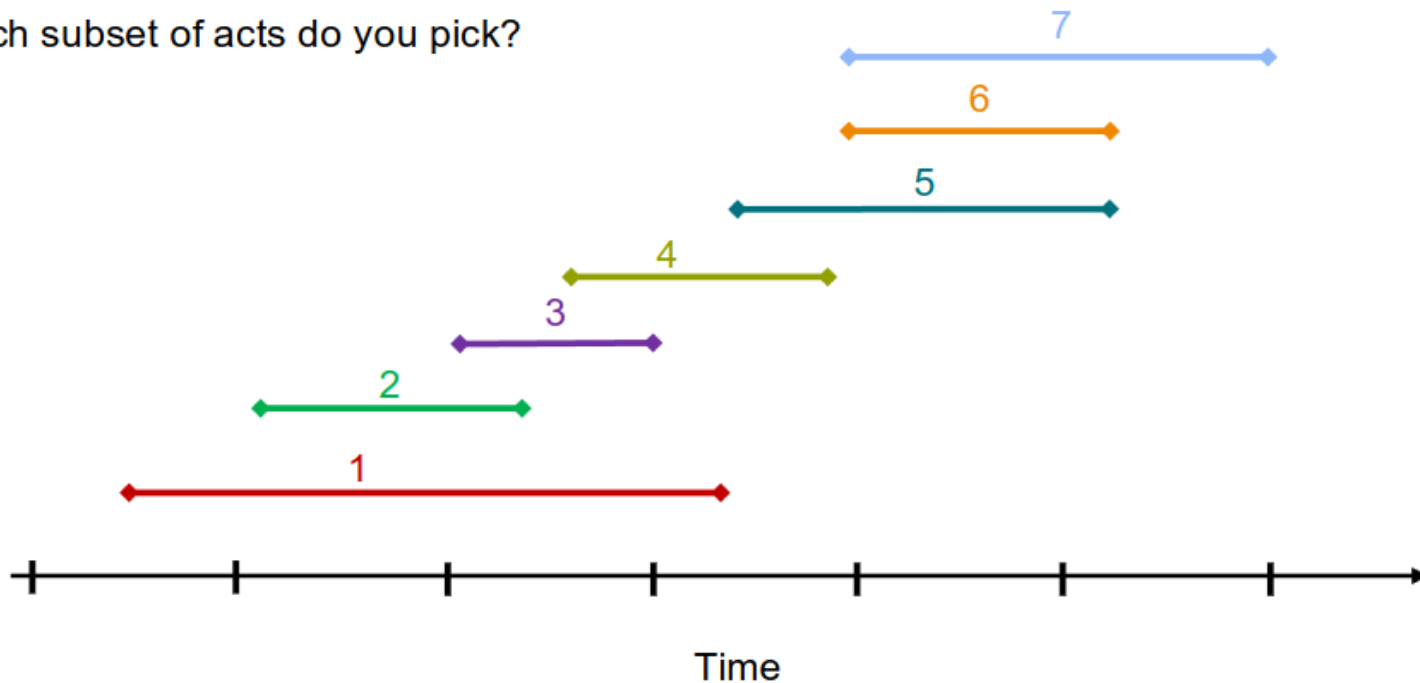
# Which is more efficient?

- ❖ For a given task, one algorithm is *more efficient* than another algorithm for that task, if the former requires fewer steps than the latter

# Interval Scheduling Problem

- ❖ Imagine you are going to a music festival with the goal of seeing as many acts as possible

Which subset of acts do you pick?



# Baseline Algorithm

- ❖ What is a simple algorithm that always correctly solves this problem?
  - Try every possible schedule and find the consistent schedule with the most items
- ❖ Why is it guaranteed to work?
  - Suppose it does not find a consistent schedule with the most items. Then, there is some consistent schedule with more items. This is a contradiction, since the baseline method checks every schedule.
- ❖ How many schedules does the baseline method need to try?

$$2^n$$

# Greedy Algorithm

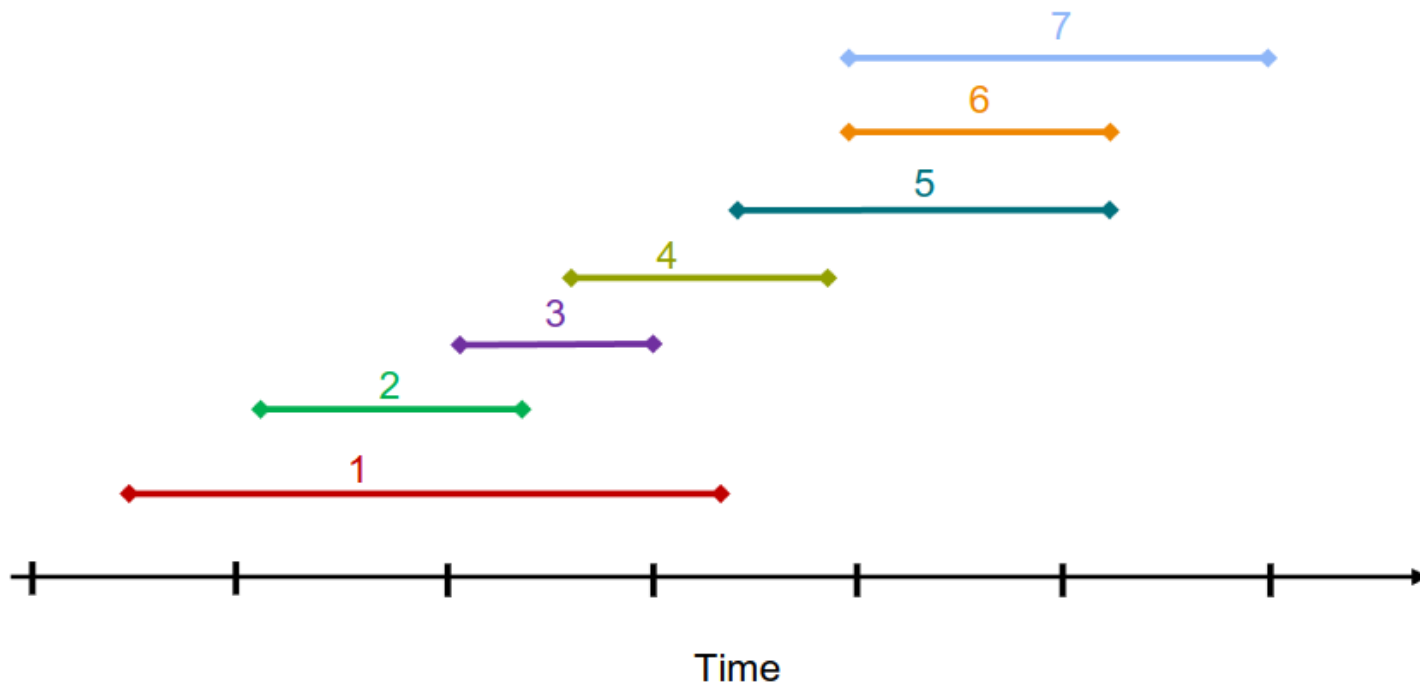
- ❖ Sort the items by \* and iteratively build schedule
- ❖ Which property should we sort the items by?

# Greedy Algorithm

- ❖ Sort the items by \* and iteratively build schedule
- ❖ Which property should we sort the items by?
  - Interval length?

# Interval Scheduling Problem

- ❖ **Goal:** find an optimal collection of acts

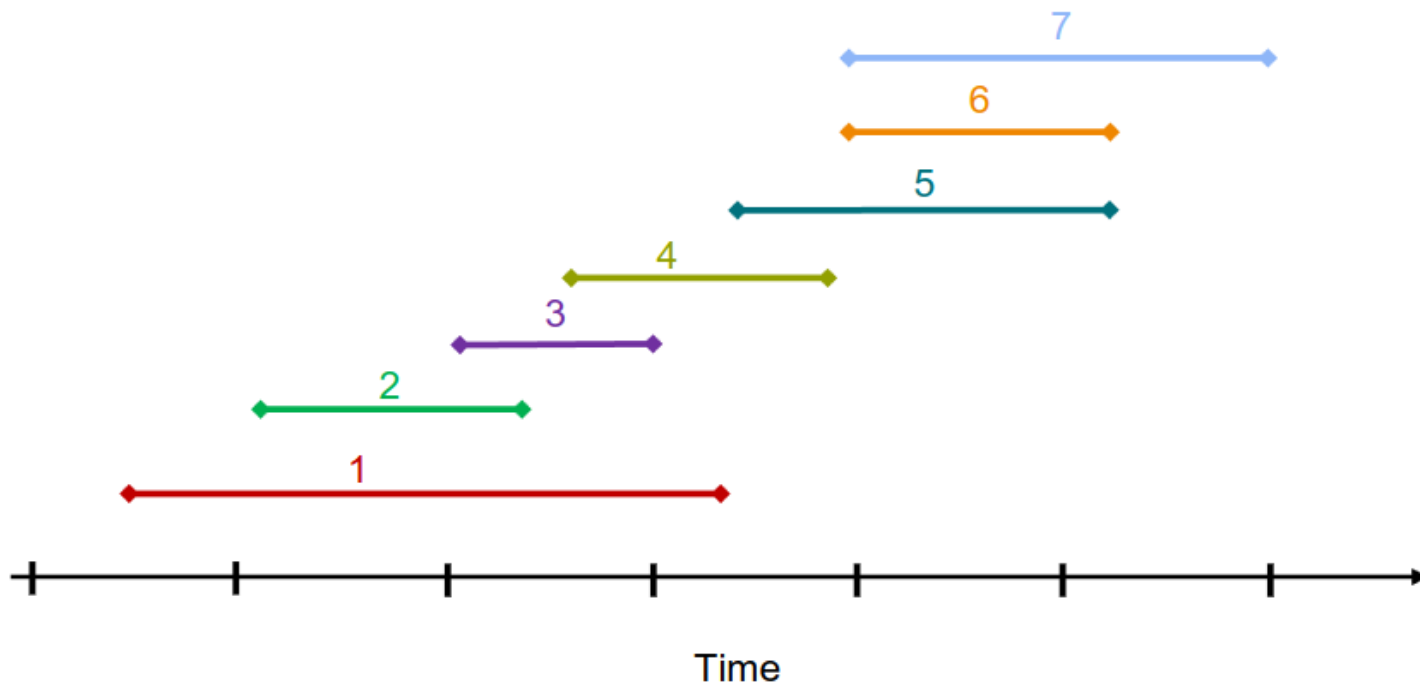


# Greedy Algorithm

- ❖ Sort the items by \* and iteratively build schedule
- ❖ Which property should we sort the items by?
  - Interval length? {3, 6}
  - Start time?

# Interval Scheduling Problem

- ❖ **Goal:** find an optimal collection of acts

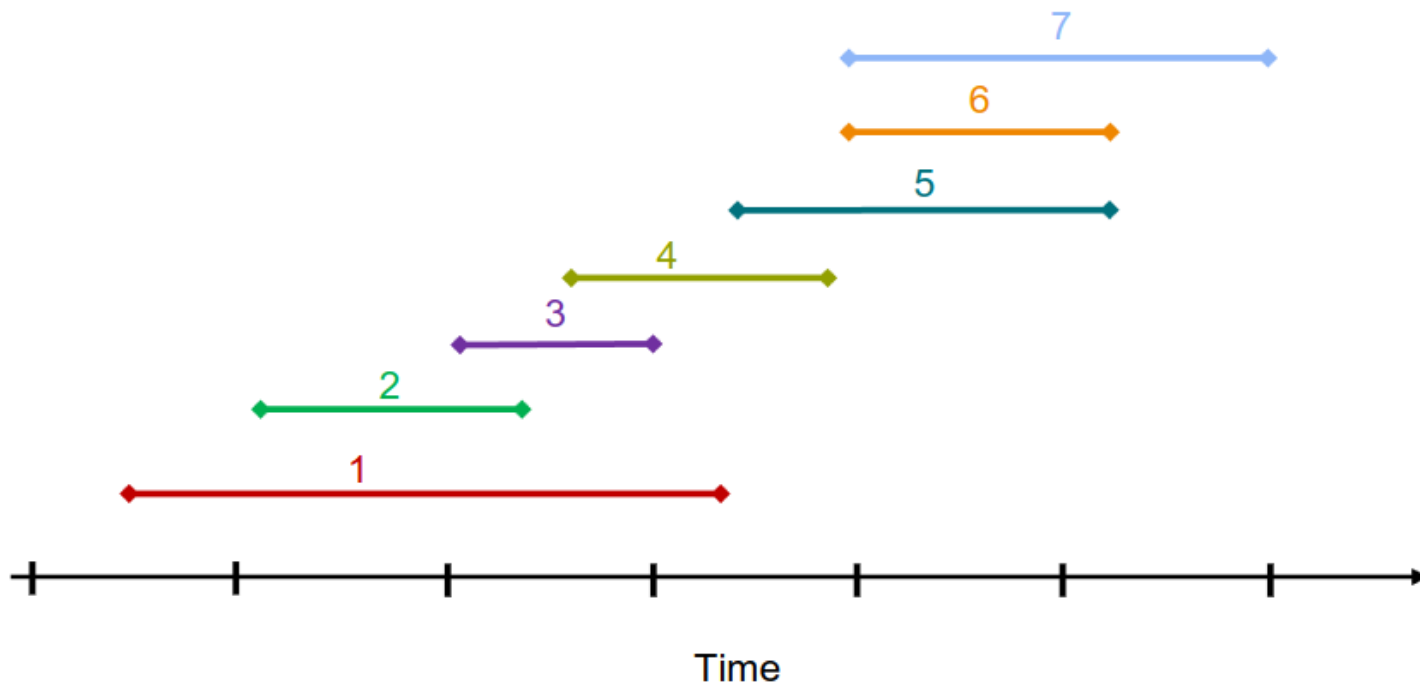


# Greedy Algorithm

- ❖ Sort the items by \* and iteratively build schedule
- ❖ Which property should we sort the items by?
  - Interval length? {3, 6}
  - Start time? {1, 5}
  - Finish time?

# Interval Scheduling Problem

- ❖ **Goal:** find an optimal collection of acts



# Greedy Algorithm

- ❖ Sort the items by \* and iteratively build schedule
- ❖ Which property should we sort the items by?
  - Interval length? {3, 6}
  - Start time? {1, 5}
  - Finish time? {2, 4, 6}

# Greedy Algorithm

- ❖ Sort the items by \* and iteratively build schedule
- ❖ Which property should we sort the items by?
  - Interval length? {3, 6}
  - Start time? {1, 5}
  - Finish time? {2, 4, 6}
- ❖ We can prove that earliest finish time is the right property for this problem

# Closing Thoughts

- ❖ How can we show that this is the right property?
- ❖ Is the baseline or the greedy algorithm more efficient for the interval scheduling problem?

# Next Time

- ❖ Empirically measuring running time